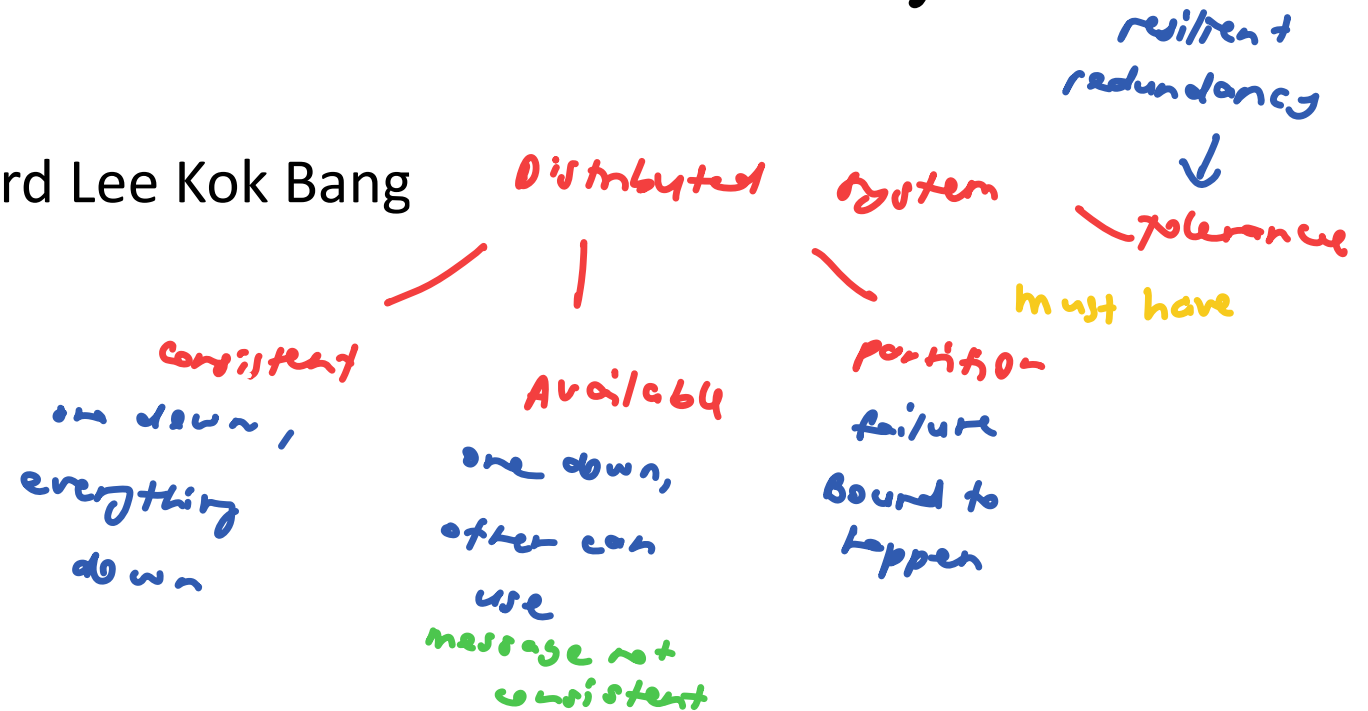


CAP Theorem

Consistency, Availability and Partition Tolerance in Distributed Systems

Bernard Lee Kok Bang



Introduction

- The CAP theorem helps system designers understand the **trade-offs** involved in designing distributed shared-data systems.
- CAP has strongly influenced the design of modern distributed databases and cloud systems.
- The theorem highlights the practical trade-offs between consistency, availability, and partition tolerance.
- CAP is also **widely misunderstood** and is often incorrectly used to classify databases

Introduction

- The CAP theorem applies to distributed systems that store and share data across multiple machines.
- Eric Brewer, at the 2000 Symposium on Principles of Distributed Computing (PODC), conjectured that in any networked shared-data system there is a fundamental trade-off between consistency, availability, and partition tolerance.
- In 2002, Seth Gilbert and Nancy Lynch of MIT published a formal proof of Brewer's conjecture.

Introduction

- Modern applications often need to scale beyond the capacity of a single machine.
- One common solution is to distribute workloads across multiple servers.
- However, distributed systems introduce new challenges such as synchronization, communication failures, and consistency management.
- The CAP theorem helps explain these trade-offs.

CAP Theorem

- The CAP theorem states that when a network partition occurs, a distributed system must choose between:
 - **Consistency**
 - **Availability**
- Partition tolerance is generally unavoidable in distributed systems.

What is CAP Theorem?

- The CAP theorem describes the limitations and trade-offs in distributed systems.
- This theorem, also known as **Brewer's theorem**, basically says that a distributed computer system cannot provide *consistency*, *availability* and *partition tolerance*, all at optimal levels. *can choose most 2*
- It states that during a network partition, a distributed system **cannot simultaneously** guarantee both strong consistency and full availability.

The three properties of CAP

- The theorem states that **networked shared-data systems** can only guarantee/strongly support two of the following **three** properties:
 - **Consistency** -
 - A guarantee that every node in a distributed cluster returns the same, most recent, successful write, or an error.
 - Consistency refers to every client having the same view of the data.
 - In CAP theorem discussions, consistency usually refers to strong consistency, where all users immediately see the latest data.
 - **Availability** -
 - **Every** non-failing node returns a response for all read and write requests in a reasonable amount of time.
 - The key word here is every.
 - **To be available**, every node on (either side of a network partition) must be able to respond in a reasonable amount of time.
 - Every request receives a non-error response, without guaranteeing it contains the most recent write

The three properties of CAP

- **Partition Tolerant** - the system continues operating despite communication failures between nodes.
- During a partition, the system must decide whether to prioritize consistency or availability.

*The term “network partitions”
-> communication breakdowns between nodes*

The Cap Theorem

- <https://tinyurl.com/bde68d77>
- <https://tinyurl.com/d9ww563t>

System Categories - CAP Theorem

- Systems are often described by how they behave during network partitions:
 - CP systems (*Consistent and Partition Tolerant*) prioritize consistency
 - AP systems (*Available and Partition Tolerant*) prioritize availability
- CA systems (*Consistent and Available*) are mostly theoretical in distributed environments

Another explanation for CP and AP

- **CP** - Consistency/Partition Tolerance –
 - Wait for a response from the partitioned node which could result in a timeout error. The system can also choose to return an error, depending on the scenario you desire. Choose Consistency over Availability when your business requirements dictate atomic reads and writes.
- **AP** - Availability/Partition Tolerance –
 - Return the most recent version of the data you have, which could be stale. This system state will also accept writes that can be processed later when the partition is resolved. Choose Availability over Consistency when your business requirements allow for some flexibility around when the data in the system synchronizes. Availability is also a compelling option when the system needs to continue to function in spite of external errors (shopping carts, etc.)

Consistent and Partition Tolerant (CP)

- prioritizes **data consistency** over availability during **network partitions**
- When a network partition occurs (nodes can't communicate with each other), the system will **refuse to respond to some requests** rather than risk returning stale or inconsistent data
- The system ensures that all nodes **always have the same view of data**, even if it means becoming temporarily unavailable to some clients
- Reads will always reflect the **most recent successful write operation**, or the operation will fail
- Use cases:
 1. Data consistency is critical (**financial transactions, inventory management**)
 2. The business logic **cannot tolerate inconsistent views of data**
 3. It's acceptable for the system to reject requests during partitions rather than return potentially incorrect data

How Partition Tolerance works in CP systems

1. **Handling network failures**: The system is designed to continue functioning correctly despite network partitions (communication breakdowns between nodes). Rather than crashing entirely, the system has mechanisms to detect and respond to these partitions.
2. **Quorum-based decisions**: Many CP systems use quorum mechanisms (like requiring responses from a majority of nodes) to maintain consistency during partitions. If a partition separates nodes into two groups (e.g., 3 nodes on one side, 2 on another), only the majority side (the 3-node group) will remain operational and able to process writes.
3. **Leader election protocols**: During partitions, CP systems often use **consensus algorithms** to elect new leaders if the current leader becomes unreachable, allowing the system to continue operation.
4. **Minority partition behavior**: Nodes in minority partitions typically become **read-only or completely unavailable**, refusing to accept write operations until the partition is resolved to prevent data inconsistencies.
5. **Automatic recovery**: When the network partition heals, these systems have mechanisms to reconcile state between previously separated nodes, bringing the minority nodes back in sync with the majority.

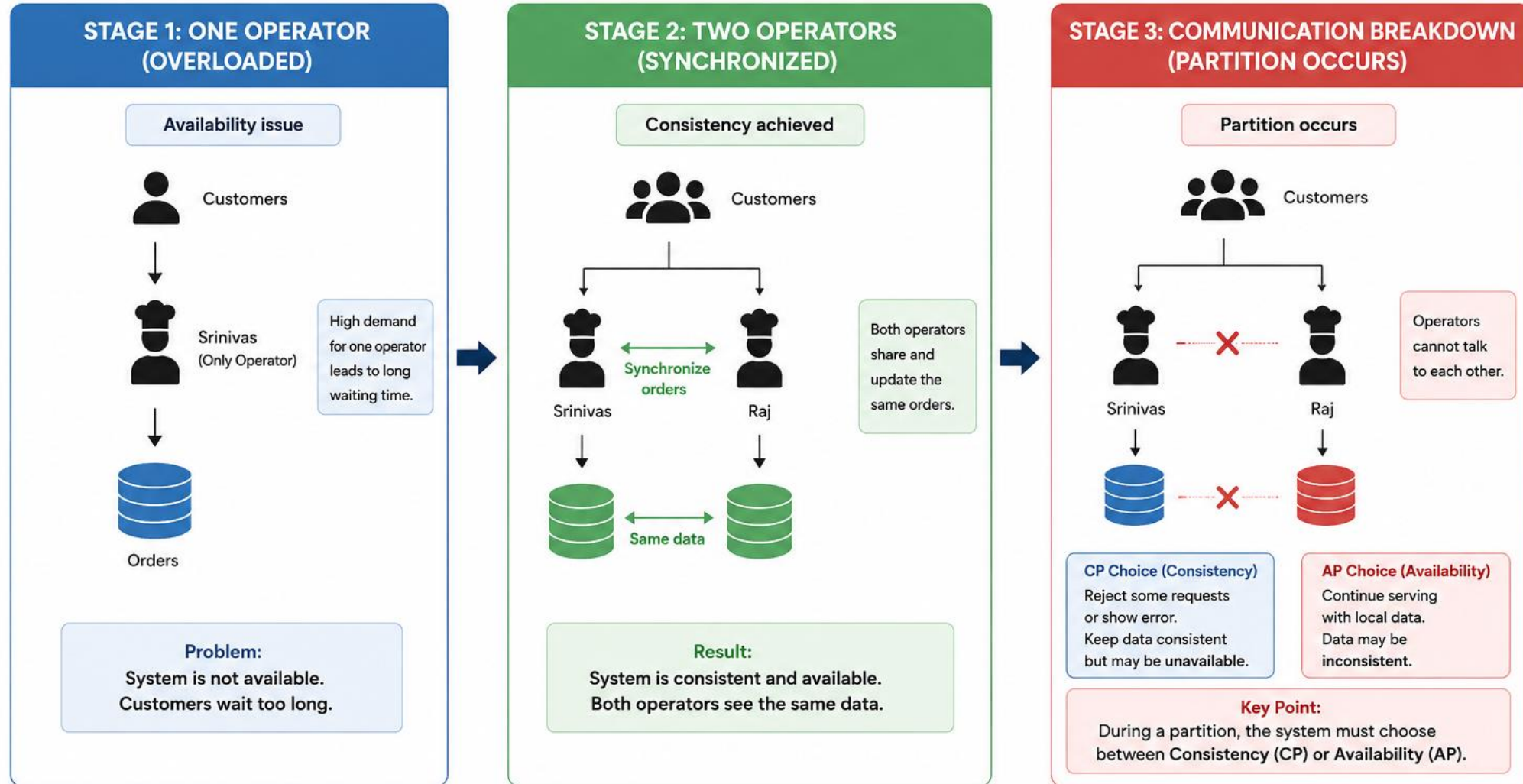
CP systems maintain a smaller but consistent operational footprint during network failures.

Summary

- The **decision** between Consistency and Availability is a *software trade off*. You can choose what to do in the face of a network partition - the control is in your hands.
- **Network outages**, both temporary and permanent, are a fact of life and occur whether you want them to or not - this exists outside of your software.
- **Building distributed systems** provide many advantages, but also adds complexity.
- **Understanding the trade-offs** available to you in the face of network errors, and choosing the right path is vital to the success of your application.
- Failing to get this right from the beginning could lead to scalability or reliability issues.

CAP THEOREM EXPLAINED WITH A RESTAURANT EXAMPLE

Three stages to understand Availability, Consistency, and Partition Tolerance



CAP Theorem in One Line:

When a network partition occurs, a distributed system must choose between Consistency and Availability.

Consistency: Everyone sees the same, most recent data.

Availability: Every request gets a response in reasonable time.

Partition Tolerance: System continues to operate despite network failures.

Partition Tolerance

- ability of different nodes or servers in a system to *continue operating* and *serving their purpose* even if there are network failures or partitions.
- even if some parts of the system cannot communicate with each other temporarily, they can *still function independently* and provide their services.
- e.g., Imagine you have a group of friends who love playing games together, and you decide to have a game night at one of your houses. However, due to some unforeseen circumstances, your house experiences a power outage, and you lose the ability to communicate with your friends.
 - partition tolerance is like the ability of your group to adapt and continue playing games despite the power outage and the temporary loss of communication between you and your friends.

Prioritizing Consistency and Partition Tolerance

- Using *Shopee* as an example
- Shopee may prioritize consistency for critical operations such as payment and inventory updates.
- During severe network failures, some operations may be delayed or temporarily unavailable to avoid inconsistent data.

Prioritizing Availability and Partition Tolerance

- Using *WhatsApp* as an example
- *Availability*
 - WhatsApp is always be up and running
 - WhatsApp would still allow you to send messages to your contacts and receive their messages
 - It would store these messages locally on your device until the network connection is restored.
 - might experience a delay in receiving messages from other contacts who are connected to a different server.
- *Partition tolerance*
 - able to handle network failures or communication breakdowns between different servers
 - WhatsApp application can continue functioning even if there are temporary disruptions in the network
- *ensure that users can always send and receive messages, regardless of network issues.*
- *messages may temporarily appear inconsistent across devices -> AP systems tolerate temporary inconsistency*